LocalScript — Локальный агент для генерации Lua-кода (MWS Octapi)

True Tech Hack 2026 · **Трек** 2

⌧ Содержание

1. О проекте
2. Зачем нужен LLM и почему не шаблоны
3. Среда выполнения Lua (MWS Octapi LowCode)
4. Архитектура системы (C1 → C2 → C3)
5. Структура файлов репозитория
6. Требования к системе
7. Быстрый старт через Docker Compose (рекомендуется)
8. Локальная разработка без Docker (Windows)
9. Тестирование и измерение качества
10. Переменные среды
11. API: краткая справка
12. Известные ограничения и подводные камни
13. Ссылки на связанные документы

1. О проекте

LocalScript — агентная система, которая принимает задачу на естественном языке (русском или английском)
и возвращает готовый валидный Lua-код для платформы автоматизации бизнес-процессов MWS Octapi LowCode.

Система работает **полностью локально**: никаких облачных API, никакого выхода в интернет во время работы.
Всё, что нужно — GPU с 8 ГБ VRAM и Docker. Единственная команда запуска:

```
docker-compose up
```

Это не NLP-поисковик и не чат-бот общего назначения. Это специализированный **генератор кода**: система обучена (через промптинг и few-shot примеры) понимать конкретные шаблоны задач платформы MWS

и превращать их в рабочий Lua-скрипт, который можно вставить непосредственно в шаг workflow-процесса.

2. Зачем нужен LLM и почему не шаблоны

Бизнес-аналитик, работающий с MWS Octapi, не пишет код — он описывает задачу словами. Одну и ту же операцию разные люди формулируют совершенно по-разному:

Формулировка	Что имеется в виду
«Оставить только выполненные заказы»	`filter(orders, status == "done")`
«Убрать из списка незавершённые заявки»	`filter(orders, status == "done")`
«Выбрать orders где status = готово»	`filter(orders, status == "done")`
«Отфильтровать заказы со статусом выполнен»	`filter(orders, status == "done")`

Шаблонный движок ищет точное совпадение строк — он не найдёт ни одного из этих вариантов. **Большая языковая модель** (LLM) понимает семантику запроса — намерение и смысл — независимо от формулировки. Именно для этого здесь нужна LLM, а не словарь шаблонов.

Кроме того, задачи на платформе могут требовать **доработки существующего кода**: «добавь проверку на nil», «если массив пустой — верни пустой массив» — и здесь статический шаблон тоже бессилен. LLM обрабатывает и такие задачи через поле `context`.

3. Среда выполнения Lua (MWS Octapi LowCode)

MWS Octapi — это платформа автоматизации бизнес-процессов (workflow engine). Бизнес-аналитики создают в ней сценарии («процессы»): последовательности шагов, условия, ветки. Каждый шаг может содержать фрагмент Lua-кода.

Это **не стандартный** Lua — это закрытая среда с ограниченным API:

Аспект	Правило
Версия Lua	5.5
Доступ к переменным	`wf.vars.X` или `wf.initVariables.X` — без JsonPath
Создание массива	`_utils.array.new()`
Пометить таблицу как массив	`_utils.array.markAsArray(arr)`
Формат ответа API	Чистый Lua-код без markdown-обёрток
Запрещённые модули	`io`, `os`, `require`, любые внешние библиотеки
Обязательное завершение	`return` statement

Примеры правильного Lua для MWS

```
-- Получить последний элемент массива
return wf.vars.emails[#wf.vars.emails]

-- Инкрементировать счётчик
return wf.vars.try_count_n + 1

-- Создать пустой массив
local result = _utils.array.new()
return result

-- Фильтрация массива по условию
local result = _utils.array.new()
for _, item in ipairs(wf.vars.orders) do
  if item.status == "done" then
    table.insert(result, item)
  end
end
return result
```

4. Архитектура системы (C1 → C2 → C3)

C1 — Контекст (что система делает в мире)

```
Пользователь / платформа MWS Octapi
        │
        │  POST /generate
        │  {"prompt": "оставить только верифицированных клиентов"}
        ▼
┌──────────────────────┐
│   LocalScript API    │
│  (этот репозиторий)  │
└──────────────────────┘
        │
        │  {"code": "local result = _utils.array.new() ..."}
        ▼
Готовый Lua-код вставляется в шаг платформы MWS Octapi
```

C2 — Контейнеры (из чего состоит система)

```
Компьютер пользователя
═══════════════════════════════════════════════════════════════════════

  ┌──────────────────────────────┐     ┌──────────────────────────────┐
  │  КОНТЕЙНЕР: ollama           │     │  КОНТЕЙНЕР: api              │
  │                              │     │                              │
  │  Образ: ollama/ollama:latest │     │  Образ: собственный Dockerfile │
  │  Порт: 11434                 │     │  Порт: 8080                  │
  │                              │     │                              │
```

```
   │  Загружает веса модели       │     │  Python + FastAPI              │
   │  qwen2.5-coder:7b-instruct   │     │  Вся бизнес-логика системы     │
   │  в GPU VRAM при старте       │◄────│  httpx POST /api/generate      │
   │                              │     │  luac для валидации кода       │
   └──────────────────────────────┘     └────────────────────────────────┘
                                                     ▲
                                                     | POST /generate {"prompt":"..."}
                                              [ Вы / тест / MWS ]
```

Контейнеры общаются через внутреннюю Docker-сеть.
Ollama доступна внутри сети по адресу `ollama:11434`, а не `localhost:11434`.

C3 — Компоненты внутри контейнера `api`

```
POST /generate {"prompt": "...", "context": "..."}
        |
        ▼
api/main.py — FastAPI endpoint
        |  Warm-up модели при старте сервера (первый запрос не будет медленным)
        ▼
api/agent.py — AgentPipeline.generate(prompt, context)
        |
        ├─ 1. api/prompt_builder.py — PromptBuilder
        |      Читает prompts/system_prompt.txt (CoT-шаблон + 20 few-shot примеров)
        |      Если есть context — добавляет его как «существующий код для доработки»
        |      Добавляет запрос пользователя
        |      → готовый промпт (~1600 токенов)
        |
        ├─ 2. OllamaClient (api/agent.py)
        |      httpx POST → ollama:11434/api/generate
        |      Модель: qwen2.5-coder:7b-instruct-q4_K_M
        |      Параметры: num_ctx=4096, temperature=0.1, num_predict=256
        |      → сырой текст ответа (включает рассуждение + код)
        |
        ├─ 3. CoT-парсер (api/agent.py)
        |      Извлекает содержимое тега <code>...</code>
        |      → чистый Lua-код без рассуждений
        |
        ├─ 4. api/validator.py — LuaValidator
        |      echo <code> | luac -o /dev/null -
        |      DRY_RUN=true → пропустить проверку (Windows без luac)
        |      → (valid: True) или (valid: False, error: "сообщение об ошибке")
        |
        └─ 5. Retry-loop (до 3 попыток)
               Если невалидно — добавить ошибку luac в промпт и повторить
               После 3 неудачных попыток — вернуть лучший вариант

        ▼
{"code": "<чистый Lua-код>"}
```

Зачем Chain-of-Thought (CoT)?

Системный промпт требует от модели сначала написать рассуждение, затем код:

```
<thinking>
1. Перевод задачи на английский
2. Пошаговый план Lua-логики с учётом MWS API
</thinking>
<code>
[только чистый Lua]
</code>
```

Записывая рассуждение **перед** кодом, модель использует эти токены как контекст
для собственной генерации — буквально «думает вслух» перед тем как писать.
Это значительно повышает точность для нестандартных запросов.
API возвращает только содержимое тега `<code>` — рассуждение остаётся внутри.

5. Структура файлов репозитория

```
task-repo/
│
├── README.md                      ← этот файл
│
├── ONBOARDING.md                   ← введение в проект с нуля (для новых участников)
│
├── Dockerfile                     ← образ Python API: python:3.11-slim + lua5.4
│
├── docker-compose.yml              ← два сервиса: api (8080) + ollama (11434)
│
├── requirements.txt                ← fastapi, uvicorn, pydantic, httpx
│
├── api/
│   ├── __init__.py                ← пустой, делает папку Python-пакетом
│   ├── main.py                    ← FastAPI app: /generate, /health, lifespan warm-
│   ├── models.py                  ← Pydantic: GenerateRequest, GenerateResponse
│   ├── agent.py                   ← AgentPipeline, OllamaClient, CoT-парсер, retry
│   ├── validator.py               ← LuaValidator: luac subprocess, DRY_RUN bypass
│   └── prompt_builder.py           ← сборка промпта: system_prompt + context + erro
│
├── prompts/
│   └── system_prompt.txt           ← CoT-шаблон + 20 few-shot примеров MWS Lua
│                                   ГЛАВНЫЙ ФАЙЛ ДЛЯ УЛУЧШЕНИЯ КАЧЕСТВА ГЕНЕРАЦИИ
│
├── docs/
│   ├── api_contract_description.md ← детальное описание HTTP API (все поля, примеры
│   ├── DEBUGGING.md                ← руководство по отладке и известные баги
│   └── architecture/              ← C4-диаграммы (по одному файлу на каждый уровень)
│       ├── c1_context.md
│       ├── c2_containers.md
│       └── c3_components.md
│
└── tests/
```

```
    ├── test_base_cases_data.py     ← словарь тест-кейсов: эталонный Lua + перефразиро
    ├── test_base_cases.py          ← unittest-раннер: тестирует API, генерирует отчёт
    └── reports/                    ← Markdown/JSON отчёты по тестам (добавлено в .gi
```

6. Требования к системе

Компонент	Минимум
GPU VRAM	8 ГБ
GPU (NVIDIA)	NVIDIA Container Toolkit для GPU passthrough в Docker
GPU (AMD)	ROCm-совместимая карта (RX 7000+), проверено на RX 7800 XT
Docker	Docker Desktop или Docker Engine + Docker Compose
Для локальной разработки	Python 3.11+, Ollama CLI

7. Быстрый старт через Docker Compose (рекомендуется)

Docker Compose — основной и рекомендуемый способ запуска.
Все компоненты запускаются одной командой. Прокси-серверы Windows (Clash Verge, Proxifier и др.)
не влияют на внутренний трафик Docker-сети.

Шаг 1 — Скачать модель (только один раз)

```
docker-compose run --rm ollama ollama pull qwen2.5-coder:7b-instruct-q4_K_M
```

Модель весит около 4.7 ГБ. После скачивания она сохраняется в Docker volume `ollama-data`
и при следующих запусках заново не скачивается.

Шаг 2 — Запустить все сервисы

```
docker-compose up --build
```

При первом старте: контейнер `api` соберётся (~1-2 минуты), затем запустится uvicorn
и выполнит warm-up модели (первый запрос к Ollama, чтобы модель загрузилась в VRAM).

Шаг 3 — Убедиться, что всё работает

```
curl -X POST http://localhost:8080/generate \
     -H "Content-Type: application/json" \
     -d '{"prompt": "получить последний email из списка"}'
```

Ожидаемый ответ:

```
{"code": "return wf.vars.emails[#wf.vars.emails]"}
```

Шаг 4 — Проверка здоровья сервиса

```
curl http://localhost:8080/health
# {"status": "ok"}
```

8. Локальная разработка без Docker (Windows)

Этот режим используется для быстрой итерации при работе над промптами и кодом.
Требует установленного Ollama и Python 3.11+.

Шаг 1 — Активировать виртуальное окружение

```
.venv\Scripts\activate
pip install -r requirements.txt
```

Шаг 2 — Запустить Ollama (отдельный терминал)

```
# Не выгружать модель из VRAM автоматически:
$env:OLLAMA_KEEP_ALIVE = "-1"
ollama serve
# Оставить этот терминал открытым
```

Шаг 3 — Проверить Ollama (в другом терминале)

```
# ВАЖНО: Если используете PowerShell 5.1, передавайте тело как UTF-8 байты (см. ниже)
Invoke-RestMethod -Uri http://localhost:11434/api/generate -Method POST `
  -ContentType "application/json" `
  -Body '{"model":"qwen2.5-coder:7b-instruct-q4_K_M","prompt":"hello","stream":fal
```

Шаг 4 — Запустить uvicorn (третий терминал)

```
$env:NO_PROXY      = "localhost,127.0.0.1,::1"
$env:DRY_RUN       = "true"   # пропустить luac на Windows
$env:OLLAMA_BASE_URL = "http://localhost:11434"
$env:OLLAMA_MODEL  = "qwen2.5-coder:7b-instruct-q4_K_M"
uvicorn api.main:app --reload --port 8080
```

Шаг 5 — Отправить тестовый запрос (PS 5.1 — UTF-8 байты)

```
$json  = '{"prompt": "отфильтровать выполненные заказы"}'
$bytes = [System.Text.Encoding]::UTF8.GetBytes($json)
Invoke-RestMethod -Uri http://localhost:8080/generate `
    -Method POST -ContentType "application/json; charset=utf-8" -Body $bytes
```

> **Почему байты, а не строка?** PowerShell 5.1 кодирует строку в теле запроса как ASCII. Кириллица превращается в `?????????`, и модель не понимает русский язык. Подробнее — в `docs/DEBUGGING.md`.

9. Тестирование и измерение качества

Тестовый набор проверяет точность генерации на перефразированных запросах.

```
# Запустить полный тест (API должен быть запущен)
python -m unittest tests.test_base_cases.TestBaseCases
```

Тест-раннер:

- Отправляет десятки вариантов одного и того же запроса к локальному API
- Сравнивает ответ с эталонным Lua-кодом (нормализованно, без пробелов)
- Выводит прогресс по каждому тест-кейсу в реальном времени
- Сохраняет подробный отчёт в `tests/reports/` в форматах `.md` и `.json`

Файл с тест-кейсами (`tests/test_base_cases_data.py`) — **единый источник правды**. Добавление нового тест-кейса: добавить запись в словарь `TEST_CASES` с ключом, ожидаемым Lua и списком перефразировок.

10. Переменные среды

Переменная	По умолчанию	Описание
`OLLAMA_BASE_URL`	`http://ollama:11434`	URL Ollama (внутри Docker: `ollama`, снаружи: `localhost`)
`OLLAMA_MODEL`	`qwen2.5-coder:7b-instruct-q4_K_M`	Имя модели
`MAX_RETRIES`	`3`	Максимум попыток retry-loop при синтаксической ошибке
`DRY_RUN`	`false`	`true` — пропустить валидацию luac (Windows без Lua)
`OLLAMA_KEEP_ALIVE`	(по умолчанию Ollama — 5 мин)	`-1` — не выгружать модель из VRAM

11. API: краткая справка

Полное описание — в `docs/api_contract_description.md`.

Endpoint	`POST /generate`
Тело запроса	`{"prompt": "...", "context": "..."}` (`context` — опционально)
Ответ	`{"code": "..."}` — чистый Lua без markdown-обёрток
Health check	`GET /health` → `{"status": "ok"}`

12. Известные ограничения и подводные камни

Системный прокси (Clash Verge, Proxifier и др.)

Python направляет ВСЕ HTTP-запросы через системный прокси, включая запросы к `localhost`. Прокси не умеет маршрутизировать loopback и возвращает `502 Bad Gateway`.

Решение: `$env:NO_PROXY = "localhost,127.0.0.1,::1"` перед запуском uvicorn.
Docker Compose эта проблема не касается.

PowerShell 5.1 и кириллица

PS 5.1 кодирует тело запроса в ASCII. Кириллица → `??????????`.
Передавайте тело как `[System.Text.Encoding]::UTF8.GetBytes(...)` или используйте PowerShell 7+.
Подробно — в `docs/DEBUGGING.md`.

DRY_RUN на Windows

`luac` недоступен в Windows из коробки. При `DRY_RUN=true` синтаксическая валидация пропускается. В Docker-контейнере `lua5.4` установлен и всё работает корректно.

Модель понимает только задачи MWS Octapi

Система настроена через промпт для задач с `wf.vars.*`. Запросы вне этой области (универсальные алгоритмы, математика и т.д.) могут дать некорректный результат.

13. Ссылки на связанные документы

Документ	Расположение	Описание
API Contract	`docs/api_contract_description.md`	Полное описание HTTP API
Debugging Guide	`docs/DEBUGGING.md`	Все известные баги и способы отладки
Onboarding	`ONBOARDING.md`	Введение для новых участников

Документ	Расположение	Описание
Architecture C4	`docs/architecture/`	C1, C2, C3 диаграммы
OpenAPI Schema	`localscript-openapi.yaml`	Машиночитаемая схема API